EXAMPLE

Compiling a String Copy Procedure, Showing How to Use C Strings

The procedure `strcpy` copies string `y` to string `x` using the null byte termination convention of C:

```
void strcpy (char x[], char y[])
{
    size t i;
    i = 0;
    while ((x[i] = y[i]) != '\0') /* copy & test byte */
    i += 1;
}
```

What is the LEGv8 assembly code?

ANSWER

Below is the basic LEGv8 assembly code segment. Assume that base addresses for arrays `x` and `y` are found in `X0` and `X1`, while `i` is in `X19`. `strcpy` adjusts the stack pointer and then saves the saved register `X19` on the stack:

```
strcpy:
    SUBI  SP,SP,#8      // adjust stack for 1 more item
    STUR  X19, [SP,#0]  // save X19
```

To initialize `i` to 0, the next instruction sets `X19` to 0 by adding 0 to 0 and placing that sum in `X19`:

```
ADD  X19,XZR,XZR // i = 0 + 0
```

This is the beginning of the loop. The address of `y[i]` is first formed by adding `i` to `y[]`:

```
L1: ADD  X10,X19,X1  // address of y[i] in X10
```

Note that we don't have to multiply `i` by 8 since `y` is an array of *bytes* and not of doublewords, as in prior examples.

To load the character in `y[i]`, we use load byte unsigned, which puts the character into `X11`:

```
LDURB  X11, [X10,#0]  // X11 = y[i]
```

A similar address calculation puts the address of `x[i]` in `X12`, and then the character in `X11` is stored at that address.

```
ADD    X12,X19,X0     // address of x[i] in X12
STURB  X11, [X12,#0]  // x[i] = y[i]
```

Next, we exit the loop if the character was 0. That is, we exit if it is the last character of the string:

```
CBZ  X11,L2  // if y[i] == 0, go to L2
```

If not, we increment `i` and loop back:

```
ADDI  X19, X19,#1  // i = i + 1
B     L1           // go to L1
```

If we don't loop back, it was the last character of the string; we restore `X19` and the stack pointer, and then return.

```
L2: LDUR  X19, [SP,#0]  // y[i] == 0: end of string.
                        // Restore old X19
    ADDI  SP,SP,#8      // pop 1 doubleword off stack
    BR    LR            // return
```

String copies usually use pointers instead of arrays in C to avoid the operations on `i` in the code above. See Section 2.14 for an explanation of arrays versus pointers.

Since the procedure `strcpy` above is a leaf procedure, the compiler could allocate `i` to a temporary register and avoid saving and restoring `X19`. Hence, instead of thinking of these registers as being just for temporaries, we can think of them as registers that the callee should use whenever convenient. When a compiler finds a leaf procedure, it exhausts all temporary registers before using registers it must save.

Characters and Strings in Java

Unicode is a universal encoding of the alphabets of most human languages. Figure 2.17 gives a list of Unicode alphabets; there are almost as many *alphabets* in Unicode as there are useful *symbols* in ASCII. To be more inclusive, Java uses Unicode for characters. By default, it uses 16 bits to represent a character.

Latin	Malayalam	Tagbanwa	General Punctuation
Greek	Sinhala	Khmer	Spacing Modifier Letters
Cyrillic	Thai	Mongolian	Currency Symbols
Armenian	Lao	Limbu	Combining Diacritical Marks
Hebrew	Tibetan	Tai Le	Combining Marks for Symbols
Arabic	Myanmar	Kangxi Radicals	Superscripts and Subscripts
Syriac	Georgian	Hiragana	Number Forms
Thaana	Hangul Jamo	Katakana	Mathematical Operators
Devanagari	Ethiopic	Bopomofo	Mathematical Alphanumeric Symbols
Bengali	Cherokee	Kanbun	Braille Patterns
Gurmukhi	Unified Canadian Aboriginal Syllabic	Shavian	Optical Character Recognition
Gujarati	Ogham	Osmanya	Byzantine Musical Symbols
Oriya	Runic	Cypriot Syllabary	Musical Symbols
Tamil	Tagalog	Tai Xuan Jing Symbols	Arrows
Telugu	Hanunoo	Yijing Hexagram Symbols	Box Drawing
Kannada	Buhid	Aegean Numbers	Geometric Shapes

FIGURE 2.17 Example alphabets in Unicode. Unicode version 4.0 has more than 160 "blocks," which is their name for a collection of symbols. Each block is a multiple of 16. For example, Greek starts at 0370_{hex}, and Cyrillic at 0400_{hex}. The first three columns show 48 blocks that correspond to human languages in roughly Unicode numerical order. The last column has 16 blocks that are multilingual and are not in order. A 16-bit encoding, called UTF-16, is the default. A variable-length encoding, called UTF-8, keeps the ASCII subset as eight bits and uses 16 or 32 bits for the other characters. UTF-32 uses 32 bits per character. To learn more, see www.unicode.org.

The LEGv8 instruction set has explicit instructions to load and store such 16-bit quantities, called *halfwords*. *Load half* (`LDURH`) loads a halfword from memory, placing it in the rightmost 16 bits of a register. Like load byte, *load half* (`LDURH`) treats the halfword as a signed number and thus sign-extends to fill the 48 leftmost bits of the register. *Store half* (`STURH`) takes a halfword from the rightmost 16 bits of a register and writes it to memory. We copy a halfword with the sequence

```
LDURH X19,[X0,#0] // Read halfword (16 bits) from source
STURH X9,[X1,#0]  // Write halfword (16 bits) to dest.
```

Strings are a standard Java class with special built-in support and predefined methods for concatenation, comparison, and conversion. Unlike C, Java includes a word that gives the length of the string, similar to Java arrays.

Elaboration: ARMv8 software is required to keep the stack aligned to "quadword" (16 byte) addresses to get better performance. This convention means that a `char` variable allocated on the stack occupies 16 bytes, even though it needs less. However, a C string variable or an array of bytes *will* pack 16 bytes per quadword, and a Java string variable or array of shorts packs 8 halfwords per quadword.

Elaboration: Reflecting the international nature of the web, most web pages today use Unicode instead of ASCII. Hence, Unicode may be even more popular than ASCII today.

Elaboration: LEGv8 keeps everything 64 bits vs. providing both 32-bit and 64-bit address instructions as in ARMv8, which means it needs to include `STURW` (store word) as an instruction even though it is not specified in ARMv8 in assembly language. ARMv8 just uses `STUR` with a W register name (32-bit register) instead of X register name (64-bit register).

Check Yourself

I. Which of the following statements about characters and strings in C and Java is true?

1. A string in C takes about half the memory as the same string in Java.
2. Strings are just an informal name for single-dimension arrays of characters in C and Java.
3. Strings in C and Java use null (0) to mark the end of a string.
4. Operations on strings, like length, are faster in C than in Java.

II. Which type of variable that can contain $1{,}000{,}000{,}000_{ten}$ takes the most memory space?

1. `long long int` in C
2. `string` in C
3. `string` in Java

2.10 LEGv8 Addressing for Wide Immediates and Addresses

Although keeping all LEGv8 instructions 32 bits long simplifies the hardware, there are times where it would be convenient to have 32-bit or larger constants or addresses. This section starts with the general solution for large constants, and then shows the optimizations for instruction addresses used in branches.

Wide Immediate Operands

Although constants are frequently short and fit into the 12-bit fields, sometimes they are bigger. The LEGv8 instruction set includes the instruction *move wide with zeros* (`MOVZ`) and *move wide with keep* (`MOVK`) specifically to set any 16 bits of a constant in a register. The former instruction zeros the rest of the bits of the register and the latter leaves the remaining bits unchanged. The 16-bit field to be loaded is specified by adding `LSL` and then the number 0, 16, 32, or 48 depending on which quadrant of the 64-bit

word is desired. These instructions allow, for example, a 32-bit constant to be created from two 32-bit instructions. Figure 2.18 shows the operation of MOVZ and MOVK.

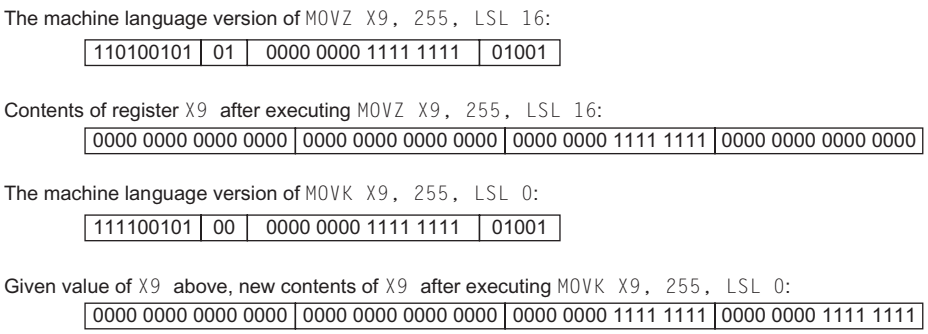


FIGURE 2.18 The effect of the MOVZ and MOVK instructions. The instruction MOVZ transfers a 16-bit immediate constant field value into one of the four quadrants leftmost of a 64-bit register, filling the other 48 bits with 0s. The instruction MOVK only changes 16 bits of the register, keeping the other bits the same.

EXAMPLE

Loading a 32-Bit Constant

What is the LEGv8 assembly code to load this 64-bit constant into register X19?

```
00000000 00000000 00000000 00000000 00000000 00111101 00001001 00000000
```

ANSWER

First, we would load bits 16 to 31 with that bit pattern, which is 61 in decimal, using MOVZ:

```
MOVZ     X19, 61, LSL 16 // 61 decimal = 0000 0000 0011 1101 binary
```

The value of register X19 afterward is:

```
00000000 00000000 00000000 00000000 00000000 00111101 00000000 00000000
```

The next step is to insert the lowest 16 bits, whose decimal value is 2304:

```
MOVK     X19, 2304, LSL 0 // 2304 decimal = 00001001 00000000
```

The final value in register X19 is the desired value:

```
00000000 00000000 00000000 00000000 00000000 00111101 00001001 00000000
```

Hardware/ Software Interface

Either the compiler or the assembler must break large constants into pieces and then reassemble them into a register. As you might expect, the immediate field's size restriction may be a problem for memory addresses in loads and stores as well as for constants in immediate instructions.

Hence, the symbolic representation of the LEGv8 machine language is no longer limited by the hardware, but by whatever the creator of an assembler chooses to include (see Section 2.12). We stick close to the hardware to explain the architecture of the computer, noting when we use the enhanced language of the assembler that is not found in the processor.

Addressing in Branches

The LEGv8 branch instructions have the simplest addressing. They use the LEGv8 instruction format, called the *B-type*, which consists of 6 bits for the operation field and the rest of the bits for the address field. Thus,

```
B    10000    // go to location 10000ten
```

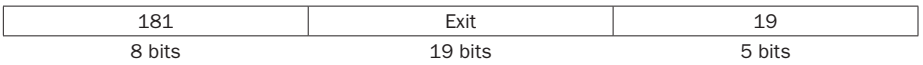
could be assembled into this format (it's actually a bit more complicated, as we will see):

where the value of the branch opcode is 5 and the branch address is 10000_{ten}.

Unlike the branch instruction, a conditional branch instruction can specify one operand in addition to the branch address. Thus,

```
CBNZ  X19, Exit  // go to Exit if X19 ≠ 0
```

is assembled into this instruction, leaving only 19 bits for the branch address:



This format is called *CB-type*, for conditional branch. (The conditional branch instructions that rely on condition codes also use the CB-type format, but they use the final field to select among the many possible branch conditions.)

If addresses of the program had to fit in this 19-bit field, it would mean that no program could be bigger than 2^{19}, which is far too small to be a realistic option today. An alternative would be to specify a register that would always

be added to the branch offset, so that a branch instruction would calculate the following:

$$\text{Program counter} = \text{Register} + \text{Branch offset}$$

This sum allows the program to be as large as 2^{64} and still be able to use conditional branches, solving the branch address size problem. Then the question is, which register?

The answer comes from seeing how conditional branches are used. Conditional branches are found in loops and in *if* statements, so they tend to branch to a nearby instruction. For example, about half of all conditional branches in SPEC benchmarks go to locations less than 16 instructions away. Since the *program counter* (PC) contains the address of the current instruction, we can branch within $\pm 2^{18}$ words of the current instruction if we use the PC as the register to be added to the address. Almost all loops and *if* statements are much smaller than 2^{18} words, so the PC is the ideal choice. This form of branch addressing is called **PC-relative addressing**.

PC-relative addressing An addressing regime in which the address is the sum of the *program counter* (PC) and a constant in the instruction.

Like most recent computers, LEGv8 uses PC-relative addressing for all conditional branches, because the destination of these instructions is likely to be close to the branch. On the other hand, branch-and-link instructions invoke procedures that have no reason to be near the call, so they normally use other forms of addressing. Hence, the LEGv8 architecture offers long addresses for procedure calls by using the B-type format for both branch and branch-and-link instructions.

Since all LEGv8 instructions are 4 bytes long, LEGv8 stretches the distance of the branch by having PC-relative addressing refer to the number of *words* to the next instruction instead of the number of bytes. Thus, the 19-bit field can branch four times as far by interpreting the field as a relative word address rather than as a relative byte address: ±1 MB from the current PC. Similarly, the 26-bit field in branch instructions is also a word address, meaning that it represents a 28-bit byte address.

The unconditional branch is also PC-relative, which means it can branch ±128 MB from the current PC.

Showing Branch Offset in Machine Language

EXAMPLE

The *while* loop on page 95 was compiled into this LEGv8 assembler code:

```
Loop:LSL  X10,X22,#3     // Temp reg X10 = 8 * i
     ADD  X10,X10,X25    // X10 = address of save[i]
     LDUR X9,[X10,#0]    // Temp reg X9 = save[i]
     SUB  X11,X9,X24     // X11 = save[i] – k
     CBNZ X11, Exit      // go to Exit if save[i] ≠ k (X11≠0)
     ADDI X22,X22,#1     // i = i + 1
     B    Loop           // go to Loop
Exit:
```

If we assume we place the loop starting at location 80000 in memory, what is the LEGv8 machine code for this loop?

The assembled instructions and their addresses are:

ANSWER

80000	1691	0	3	22	10
80004	1112	25	0	10	10
80008	1896	0	0	10	9
80012	1624	24	0	9	11
80016	181	3			11
80020	580	1		22	22
80024	5	−6			
80028	...				

Remember that LEGv8 instructions have byte addresses, so addresses of sequential words differ by 4, the number of bytes in a word, and that branches multiply their address fields by 4, the size of LEGv8 instructions in bytes. The `CBNZ` instruction on the fifth line adds 3 words or 12 bytes to the address of the instruction, specifying the branch destination relative to the branch instruction (12 + 80016) and not using the full destination address (80028). The branch instruction on the last line does a similar calculation for a backwards branch (−24 + 80024), corresponding to the label `Loop`.

Hardware/ Software Interface

Most conditional branches are to a nearby location, but occasionally they branch far away, farther than can be represented in the 19 bits of the conditional branch instruction. The assembler comes to the rescue just as it did with large addresses or constants: it inserts an unconditional branch to the branch target, and inverts the condition so that the conditional branch decides whether to skip the unconditional branch.

EXAMPLE

Branching Far Away

Given a branch on register `X19` being equal to register zero,

```
CBZ    X19, L1
```

ANSWER

replace it by a pair of instructions that offers a much greater branching distance. These instructions replace the short-address conditional branch:

```
       CBNZ    X19, L2
       B       L1
L2:
```

LEGv8 Addressing Mode Summary

addressing mode One of several addressing regimes delimited by their varied use of operands and/or addresses.

Multiple forms of addressing are generically called **addressing modes**. Figure 2.19 shows how operands are identified for each addressing mode. The addressing modes of the LEGv8 instructions are the following:

1. *Immediate addressing*, where the operand is a constant within the instruction itself.
2. *Register addressing*, where the operand is a register.
3. *Base* or *displacement addressing*, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction.
4. *PC-relative addressing*, where the branch address is the sum of the PC and a constant in the instruction.

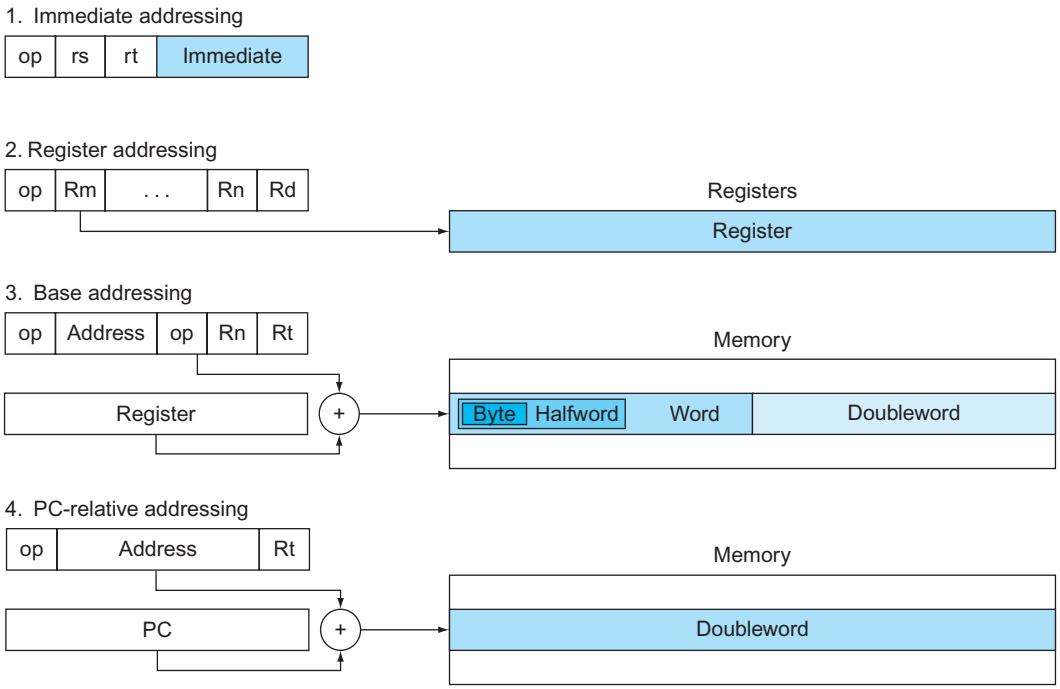


FIGURE 2.19 Illustration of four LEGv8 addressing modes. The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, halfwords, words, or doublewords. For mode 1, the operand is part of the instruction itself. Mode 4 addresses instructions in memory, with mode 4 adding a long address shifted left 2 bits to the PC. Note that a single operation can use more than one addressing mode. Add, for example, uses both immediate (`ADDI`) and register (`ADD`) addressing.

Decoding Machine Language

Sometimes you are forced to reverse-engineer machine language to create the original assembly language. One example is when looking at "core dump." Figure 2.20 shows the LEGv8 encoding of the opcodes for the LEGv8 machine language. This figure helps when translating by hand between assembly language and machine language.

Instruction	Opcode	Opcode Size	11-bit opcode range		Instruction Format
			Start	End	
B	000101	6	160	191	B - format
STURB	00111000000	11	448		D - format
LDURB	00111000010	11	450		D - format
B.cond	01010100	8	672	679	CB - format
ORRI	1011001000	10	712	713	I - format
EORI	1101001000	10	840	841	I - format
STURH	01111000000	11	960		D - format
LDURH	01111000010	11	962		D - format
AND	10001010000	11	1104		R - format
ADD	10001011000	11	1112		R - format
ADDI	1001000100	10	1160	1161	I - format
ANDI	1001001000	10	1168	1169	I - format
BL	100101	6	1184	1215	B - format
ORR	10101010000	11	1360		R - format
ADDS	10101011000	11	1368		R - format
ADDIS	1011000100	10	1416	1417	I - format
CBZ	10110100	8	1440	1447	CB - format
CBNZ	10110101	8	1448	1455	CB - format
STURW	10111000000	11	1472		D - format
LDURSW	10111000100	11	1476		D - format
STXR	11001000000	11	1600		D - format
LDXR	11001000010	11	1602		D - format
EOR	11101010000	11	1616		R - format
SUB	11001011000	11	1624		R - format
SUBI	1101000100	10	1672	1673	I - format
MOVZ	110100101	9	1684	1687	IM - format
LSR	11010011010	11	1690		R - format
LSL	11010011011	11	1691		R - format
BR	11010110000	11	1712		R - format
ANDS	11101010000	11	1872		R - format
SUBS	11101011000	11	1880		R - format
SUBIS	1111000100	10	1928	1929	I - format
ANDIS	1111001000	10	1936	1937	I - format
MOVK	111100101	9	1940	1943	IM - format
STUR	11111000000	11	1984		D - format
LDUR	11111000010	11	1986		D - format

FIGURE 2.20 LEGv8 instruction encoding. The varying size opcode values can be mapped into the space they occupy in the widest opcodes. By looking at the first 11 bits of the instruction and looking up the value, you can see which instruction it refers to.

Decoding Machine Code

EXAMPLE

What is the assembly language statement corresponding to this machine instruction?

$8b0f0013_{hex}$

ANSWER

The first step is converting hexadecimal to binary:

```
1000 1011 0000 1111 0000 0000 0001 0011
```

To know how to interpret the bits, we need to determine the instruction format, and to do that we first need to find the opcode field. The problem is that the opcode varies from 6 bits to 11 bits depending on the format. Since opcodes must be unique, one way to identify them is to see how many 11-bit opcodes do the shorter opcodes correspond.

For example, the branch instruction `B` use a 6-bit opcode with the value:

```
00 0101
```

Measured in 11-bit opcodes, it occupies all the opcode values from

```
00 0101 00000
```

to

```
00 0101 11111
```

That is, if any 11-bit opcode had a value in that range, such as

```
00 0101 00100
```

it would conflict with the 6-bit opcode of branch.

Figure 2.20 lists the instructions in LEGv8 in numerical order by opcode, showing the range of the 11-bit opcode space they occupy. For example, branch goes from 160 to 191, which is the decimal version of the bit patterns above. To determine the opcode, you just take the first 11 bits of the instruction, covert it to decimal representation, and then look up the table to find the instruction and its format.

In this example, the tentative opcode field is then 10001011000_{two}, which is 1112_{ten}. When we search Figure 2.20, we see that opcode corresponds to the `ADD` instruction, which uses the R-format. Thus, we can parse the binary format into fields listed in Figure 2.21:

opcode	Rm	shamt	Rn	Rd
10001011000	00101	000000	01111	10000

We decode the rest of the instruction by looking at the field values. The decimal values are 5 for the Rm field, 15 for Rn, and 16 for Rd (shamt is unused). These numbers represent registers `X5`, `X15`, and `X16`. Now we can reveal the assembly instruction:

```
ADD X16,X15,X5
```

Figure 2.21 shows all the LEGv8 instruction formats. Figure 2.1 on pages 64–65 shows the LEGv8 assembly language revealed in this chapter. The next chapter covers LEGv8 instructions for multiply, divide, and arithmetic for real numbers.

Name	Fields							Comments
Field size		6 to 11 bits	5 to 10 bits	5 or 4 bits	2 bits	5 bits	5 bits	All LEGv8 instructions are 32 bits long
R-format	R	opcode	Rm	shamt		Rn	Rd	Arithmetic instruction format
I-format	I	opcode	immediate			Rn	Rd	Immediate format
D-format	D	opcode	address		op2	Rn	Rt	Data transfer format
B-format	B	opcode	address					Unconditional Branch format
CB-format	CB	opcode	address				Rt	Conditional Branch format
IW-format	IW	opcode	immediate				Rd	Wide Immediate format

FIGURE 2.21 LEGv8 instruction formats.

Check Yourself

I. What is the range of addresses for conditional branches in LEGv8 (K = 1024)?

1. Addresses between 0 and 512K − 1
2. Addresses between 0 and 2048K − 1
3. Addresses up to about 256K before the branch to about 256K after
4. Addresses up to about 1024K before the branch to about 1024K after

II. What is the range of addresses for branch and branch and link in LEGv8 (M = 1024K)?

1. Addresses between 0 and 64M − 1
2. Addresses between 0 and 256M − 1
3. Addresses up to about 32M before the branch to about 32M after
4. Addresses up to about 128M before the branch to about 128M after

Elaboration: An easy-to-understand way for hardware to figure out the format of the instruction in parallel is to think of the hardware as having a read-only memory whose address size matches the largest opcode and whose content tells the hardware what to do for the specific instruction. Thus, instructions like *add* (ADD) that have an 11-bit opcode have a single entry in the memory, but instructions like *branch* (B) with a 6-bit opcode have many redundant copies. In fact, B has $2^{11}/2^{6}=2^{5}$ or 32 entries. A more efficient hardware structure than a read-only memory that will accomplish the same task is a *programmable-logic array* (PLA), which essentially modifies the address decoder so that there is a single entry for every opcode, no matter its size (see Appendix B).

2.11 Parallelism and Instructions: Synchronization

Parallel execution is easier when tasks are independent, but often they need to cooperate. Cooperation usually means some tasks are writing new values that others must read. To know when a task is finished writing so that it is safe for another to read, the tasks need to synchronize. If they don't synchronize, there is a danger of a **data race**, where the results of the program can change depending on how events happen to occur.

data race Two memory accesses form a data race if they are from different threads to the same location, at least one is a write, and they occur one after another.

For example, recall the analogy of the eight reporters writing a story on pages 44–45 of Chapter 1. Suppose one reporter needs to read all the prior sections before writing a conclusion. Hence, he or she must know when the other reporters have finished their sections, so that there is no danger of sections being changed afterwards. That is, they had better synchronize the writing and reading of each section so that the conclusion will be consistent with what is printed in the prior sections.

In computing, synchronization mechanisms are typically built with user-level software routines that rely on hardware-supplied synchronization instructions. In this section, we focus on the implementation of *lock* and *unlock* synchronization operations. Lock and unlock can be used straightforwardly to create regions where only a single processor can operate, called a *mutual exclusion*, as well as to implement more complex synchronization mechanisms.

The critical ability we require to implement synchronization in a multiprocessor is a set of hardware primitives with the ability to *atomically* read and modify a memory location. That is, nothing else can interpose itself between the read and the write of the memory location. Without such a capability, the cost of building basic synchronization primitives will be high and will increase unreasonably as the processor count increases.

There are a number of alternative formulations of the basic hardware primitives, all of which provide the ability to atomically read and modify a location, together with some way to tell if the read and write were performed atomically. In general, architects do not expect users to employ the basic hardware primitives, but instead expect system programmers will use the primitives to build a synchronization library, a process that is often complex and tricky.

Let's start with one such hardware primitive and show how it can be used to build a basic synchronization primitive. One typical operation for building synchronization operations is the *atomic exchange* or *atomic swap*, which interchanges a value in a register for a value in memory.

To see how to use this to build a basic synchronization primitive, assume that we want to build a simple lock where the value 0 is used to indicate that the lock is free and 1 is used to indicate that the lock is unavailable. A processor tries to set the lock by doing an exchange of 1, which is in a register, with the memory address corresponding to the lock. The value returned from the exchange instruction is 1 if some other processor had already claimed access, and 0 otherwise. In the latter case, the value is also changed to 1, preventing any competing exchange in another processor from also retrieving a 0.

For example, consider two processors that each try to do the exchange simultaneously: this race is prevented, since exactly one of the processors will perform the exchange first, returning 0, and the second processor will return 1 when it does the exchange. The key to using the exchange primitive to implement synchronization is that the operation is atomic: the exchange is indivisible, and two simultaneous exchanges will be ordered by the hardware. It is impossible for two processors trying to set the synchronization variable in this manner to both think they have simultaneously set the variable.

Implementing a single atomic memory operation introduces some challenges in the design of the processor, since it requires both a memory read and a write in a single, uninterruptible instruction.

An alternative is to have a pair of instructions in which the second instruction returns a value showing whether the pair of instructions was executed as if the pair was atomic. The pair of instructions is effectively atomic if it appears as if all other operations executed by any processor occurred before or after the pair. Thus, when an instruction pair is effectively atomic, no other processor can change the value between the pair of instructions.

In LEGv8 this pair of instructions includes a special load called a *load exclusive register* (`LDXR`) and a special store called a *store exclusive register* (`STXR`). These instructions are used in sequence: if the contents of the memory location specified by the load exclusive are changed before the store exclusive to the same address occurs, then the store exclusive fails and does not write the value to memory. The store exclusive is defined to both store the value of a (presumably different) register in memory *and* to change the value of another register to a 0 if it succeeds and to a 1 if it fails. Thus, `STXR` specifies three registers: one to hold the address, one to indicate whether the atomic operation failed or succeeded, and one to hold the value to be stored in memory if it succeeded. Since the load exclusive returns the initial value, and the store exclusive returns 0 only if it succeeds, the following sequence implements an atomic exchange on the memory location specified by the contents of `X20`:

```
again:LDXR X10,[X20,#0]     // load exclusive
      STXR X23, X9, [X20]   // store exclusive
      CBNZ X9,again         // branch if store fails
      ADD  X23,XZR,X10      // put loaded value in X23
```

Any time a processor intervenes and modifies the value in memory between the LDXR and STXR instructions, the STXR returns 1 in X9, causing the code sequence to try again. At the end of this sequence, the contents of X23 and the memory location specified by X20 have been atomically exchanged.

Elaboration: Although it was presented for multiprocessor synchronization, atomic exchange is also useful for the operating system in dealing with multiple processes in a single processor. To make sure nothing interferes in a single processor, the store exclusive also fails if the processor does a context switch between the two instructions (see Chapter 5).

Elaboration: An advantage of the load/store exclusive mechanism is that it can be used to build other synchronization primitives, such as *atomic compare and swap* or *atomic fetch-and-increment*, which are used in some parallel programming models. These involve more instructions between the LDXR and the STXR, but not too many.

Since the store exclusive will fail after either another attempted store to the load exclusive address or any exception, care must be taken in choosing which instructions are inserted between the two instructions. In particular, only register–register instructions can safely be permitted; otherwise, it is possible to create deadlock situations where the processor can never complete the STXR because of repeated page faults. In addition, the number of instructions between the load exclusive and the store exclusive should be small to minimize the probability that either an unrelated event or a competing processor causes the store exclusive to fail frequently.

Elaboration: While the code above implemented an atomic exchange, the following code would more efficiently acquire a lock at the location in register X20, where the value of 0 means the lock was free and 1 to mean lock was acquired:

```
        ADDI X11,XZR,#1        // copy locked value
again: LDXR X10,[X20,#0]       // load exclusive to read lock
        CBNZ X10, again        // check if it is 0 yet
        STXR X11, X9, [X20]    // attempt to store new value
        BNEZ X9,again          // branch if store fails
```

We release the lock just using a regular store to write 0 into the location:

```
        STUR XZR, [X20,#0]     // free lock by writing 0
```

Check Yourself

When do you use primitives like load exclusive and store exclusive?

1. When cooperating threads of a parallel program need to synchronize to get proper behavior for reading and writing shared data.
2. When cooperating processes on a uniprocessor need to synchronize for reading and writing shared data.

2.12 Translating and Starting a Program

This section describes the four steps in transforming a C program in a file from storage (disk or flash memory) into a program running on a computer. Figure 2.22 shows the translation hierarchy. Some systems combine these steps to reduce translation time, but programs go through these four logical phases. This section follows this translation hierarchy.

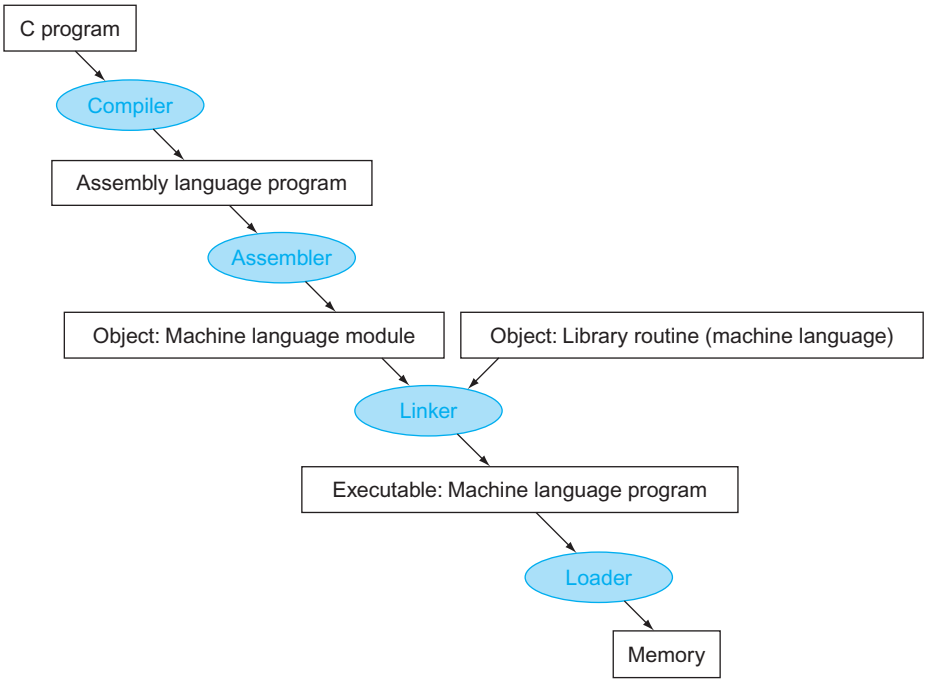


FIGURE 2.22 A translation hierarchy for C. A high-level language program is first compiled into an assembly language program and then assembled into an object module in machine language. The linker combines multiple modules with library routines to resolve all references. The loader then places the machine code into the proper memory locations for execution by the processor. To speed up the translation process, some steps are skipped or combined. Some compilers produce object modules directly, and some systems use linking loaders that perform the last two steps. To identify the type of file, UNIX follows a suffix convention for files: C source files are named `x.c`, assembly files are `x.s`, object files are named `x.o`, statically linked library routines are `x.a`, dynamically linked library routes are `x.so`, and executable files by default are called `a.out`. MS-DOS uses the suffixes `.C`, `.ASM`, `.OBJ`, `.LIB`, `.DLL`, and `.EXE` to the same effect.

Compiler

The compiler transforms the C program into an *assembly language program*, a symbolic form of what the machine understands. High-level language programs take many fewer lines of code than assembly language, so programmer productivity is much higher.

In 1975, many operating systems and assemblers were written in **assembly language** because memories were small and compilers were inefficient. The million-fold increase in memory capacity per single DRAM chip has reduced program size concerns, and optimizing compilers today can produce assembly language programs nearly as well as an assembly language expert, and sometimes even better for large programs.

assembly language A symbolic language that can be translated into binary machine language.

Assembler

Since assembly language is an interface to higher-level software, the assembler can also treat common variations of machine language instructions as if they were instructions in their own right. The hardware need not implement these instructions; however, their appearance in assembly language simplifies translation and programming. Such instructions are called **pseudoinstructions**.

pseudoinstruction A common variation of assembly language instructions often treated as if it were an instruction in its own right.

As mentioned above, the LEGv8 hardware makes sure that register `XZR (X31)` always has the value 0. That is, whenever register `XZR` is used, it supplies a 0, and the programmer cannot change the value of register `XZR`. Register `XZR` is used to create the assembly language instruction that copies the contents of one register to another. Thus, the LEGv8 assembler accepts the following instruction even though it is not found in the LEGv8 machine language:

```
MOV X9,X10       // register X9 gets register X10
```

The assembler converts this assembly language instruction into the machine language equivalent of the following instruction:

```
ORR X9,XZR,X10 // register X9 gets 0 OR register X10
```

The LEGv8 assembler also converts `CMP` (compare) into a subtract instruction that sets the condition codes and has `XZR` as the destination. Thus

```
CMP X9,X10  // compare X9 to X10 and set condition codes
```

becomes

```
SUBS XZR,X9,X10  // use X9 – X10 to set condition codes
```

It also converts branches to faraway locations into two branches. As mentioned above, the LEGv8 assembler allows large constants to be loaded into a register despite the limited size of the immediate instructions. Thus, the assembler can accept *load address* (`LDA`) and turn it into the necessary instruction sequence. Finally, it can simplify

the instruction set by determining which variation of an instruction the programmer wants. For example, the LEGv8 assembler does not require the programmer to specify the immediate version of the instruction when using a constant for arithmetic and logical instructions; it just generates the proper opcode. Thus

```
AND X9,X10,#15 // register X9 gets X10 AND 15
```

becomes

```
ANDI X9,X10,#15 // register X9 gets X10 AND 15
```

We include the "I" on the instructions to remind the reader that this instruction produces a different opcode in a different instruction format than the `AND` instruction with no immediate operands.

In summary, pseudoinstructions give LEGv8 a richer set of assembly language instructions than those implemented by the hardware. If you are going to write assembly programs, use pseudoinstructions to simplify your task. To understand the LEGv8 architecture and be sure to get best performance, however, study the real LEGv8 instructions found in Figures 2.1 and 2.20.

Assemblers will also accept numbers in a variety of bases. In addition to binary and decimal, they usually accept a base that is more succinct than binary yet converts easily to a bit pattern. LEGv8 assemblers use hexadecimal.

Such features are convenient, but the primary task of an assembler is assembly into machine code. The assembler turns the assembly language program into an *object file*, which is a combination of machine language instructions, data, and information needed to place instructions properly in memory.

To produce the binary version of each instruction in the assembly language program, the assembler must determine the addresses corresponding to all labels. Assemblers keep track of labels used in branches and data transfer instructions in a **symbol table**. As you might expect, the table contains pairs of symbols and addresses.

symbol table A table that matches names of labels to the addresses of the memory words that instructions occupy.

The object file for UNIX systems typically contains six distinct pieces:

- The *object file header* describes the size and position of the other pieces of the object file.
- The *text segment* contains the machine language code.
- The *static data segment* contains data allocated for the life of the program. (UNIX allows programs to use both *static data*, which is allocated throughout the program, and *dynamic data*, which can grow or shrink as needed by the program. See Figure 2.14.)
- The *relocation information* identifies instructions and data words that depend on absolute addresses when the program is loaded into memory.
- The *symbol table* contains the remaining labels that are not defined, such as external references.

- The *debugging information* contains a concise description of how the modules were compiled so that a debugger can associate machine instructions with C source files and make data structures readable.

The next subsection shows how to attach such routines that have already been assembled, such as library routines.

Elaboration: Similar to the case of `ADD` and `ADDI` mentioned above, the full ARMv8 instruction set does not use `ANDI` when one of the operands is an immediate; it just uses `AND`, and lets the assembler pick the proper opcode. For teaching purposes, LEGv8 again distinguishes the two cases with different mnemonics.

Linker

What we have presented so far suggests that a single change to one line of one procedure requires compiling and assembling the whole program. Complete retranslation is a terrible waste of computing resources. This repetition is particularly wasteful for standard library routines, because programmers would be compiling and assembling routines that by definition almost never change. An alternative is to compile and assemble each procedure independently, so that a change to one line would require compiling and assembling only one procedure. This alternative requires a new systems program, called a **link editor** or **linker**, which takes all the independently assembled machine language programs and "stitches" them together. The reason a linker is useful is that it is much faster to patch code than it is to recompile and reassemble.

linker Also called **link editor**. A systems program that combines independently assembled machine language programs and resolves all undefined labels into an executable file.

There are three steps for the linker:

1. Place code and data modules symbolically in memory.
2. Determine the addresses of data and instruction labels.
3. Patch both the internal and external references.

The linker uses the relocation information and symbol table in each object module to resolve all undefined labels. Such references occur in branch instructions and data addresses, so the job of this program is much like that of an editor: it finds the old addresses and replaces them with the new addresses. Editing is the origin of the name "link editor," or linker for short.

If all external references are resolved, the linker next determines the memory locations each module will occupy. Recall that Figure 2.14 on page 108 shows the LEGv8 convention for allocation of program and data to memory. Since the files were assembled in isolation, the assembler could not know where a module's instructions and data would be placed relative to other modules. When the linker places a module in memory, all *absolute* references, that is, memory addresses that are not relative to a register, must be *relocated* to reflect its true location.

The linker produces an **executable file** that can be run on a computer. Typically, this file has the same format as an object file, except that it contains no unresolved references. It is possible to have partially linked files, such as library routines, that still have unresolved addresses and hence result in object files.

executable file A functional program in the format of an object file that contains no unresolved references. It can contain symbol tables and debugging information. A "stripped executable" does not contain that information. Relocation information may be included for the loader.

EXAMPLE

Linking Object Files

Link the two object files below. Show updated addresses of the first few instructions of the completed executable file. We show the instructions in assembly language just to make the example understandable; in reality, the instructions would be numbers.

Note that in the object files we have highlighted the addresses and symbols that must be updated in the link process: the instructions that refer to the addresses of procedures A and B and the instructions that refer to the addresses of data doublewords X and Y.

Object file header			
	Name	Procedure A	
	Text size	100_{hex}	
	Data size	20_{hex}	
Text segment	Address	Instruction	
	0	LDUR X0, [X27,#0]	
	4	BL 0	
	. . .	. . .	
Data segment	0	(X)	
	. . .	. . .	
Relocation information	Address	Instruction type	Dependency
	0	LDUR	X
	4	BL	B
Symbol table	Label	Address	
	X	–	
	B	–	
	Name	Procedure B	
	Text size	200_{hex}	
	Data size	30_{hex}	
Text segment	Address	Instruction	
	0	STUR X1, [X27,#0]	
	4	BL 0	
	. . .	. . .	
Data segment	0	(Y)	
	. . .	. . .	
Relocation information	Address	Instruction type	Dependency
	0	STUR	Y
	4	BL	A
Symbol table	Label	Address	
	Y	–	
	A	–	

ANSWER

Procedure A needs to find the address for the variable labeled X to put in the load instruction and to find the address of procedure B to place in the BL instruction. Procedure B needs the address of the variable labeled Y for the store instruction and the address of procedure A for its BL instruction.

From Figure 2.14 on page 108, we know that the text segment starts at address $0000\ 0000\ 0040\ 0000_{hex}$ and the data segment at $0000\ 0000\ 1000\ 0000_{hex}$. The text of procedure A is placed at the first address and its data at the second. The object file header for procedure A says that its text is 100_{hex} bytes and its data is 20_{hex} bytes, so the starting address for procedure B text is $40\ 0100_{hex}$, and its data starts at $1000\ 0020_{hex}$.

Executable file header		
	Text size	300_{hex}
	Data size	50_{hex}
Text segment	Address	Instruction
	$0000\ 0000\ 0040\ 0000_{hex}$	LDUR X0, [X27,#0_{hex}]
	$0000\ 0000\ 0040\ 0004_{hex}$	BL 000 00FC$_{hex}$
	. . .	. . .
	$0000\ 0000\ 0040\ 0100_{hex}$	STUR X1, [X27,#20_{hex}]
	$0000\ 0000\ 0040\ 0104_{hex}$	BL 3FF FEFC$_{hex}$
	. . .	. . .
Data segment	Address	
	$0000\ 0000\ 1000\ 0000_{hex}$	(X)
	. . .	. . .
	$0000\ 0000\ 1000\ 0020_{hex}$	(Y)
	. . .	. . .

Now the linker updates the address fields of the instructions. It uses the instruction type field to know the format of the address to be edited. We have two types here:

1. The branch and link instructions use PC-relative addressing. Thus, for the BL at address $40\ 0004_{hex}$ to go to $40\ 0100_{hex}$ (the address of procedure B), it must put ($40\ 0100_{hex}$ – $40\ 0004_{hex}$) or $000\ 00FC_{hex}$ in its address field. Similarly, since $40\ 0000_{hex}$ is the address of procedure A, the BL at $40\ 0104_{hex}$ gets the negative number $3FF\ FEFC_{hex}$ ($40\ 0000_{hex}$ – $40\ 0104_{hex}$) in its address field.

2. The load and store addresses are harder because they are relative to a base register. This example uses X27 as the base register, assuming it is initialized to $0000\ 0000\ 1000\ 0000_{hex}$. To get the address $0000\ 0000\ 1000\ 0000_{hex}$ (the address of doubleword X), we place 0_{hex} in the address field of LDUR at address $40\ 0000_{hex}$. Similarly, we place 20_{hex} in the address field of STUR at address $40\ 0100_{hex}$ to get the address $0000\ 0000\ 1000\ 0020_{hex}$ (the address of doubleword Y).

Elaboration: Recall that LEGv8 instructions are word aligned, so `BL` drops the right two bits to increase the instruction's address range. Thus, it uses 26 bits to create a 28-bit byte address. Hence, the actual address in the lower 26 bits of the first `BL` instruction in this example is $000\ 003F_{hex}$, rather than $000\ 00FC_{hex}$.

Loader

loader A systems program that places an object program in main memory so that it is ready to execute.

Now that the executable file is on disk, the operating system reads it to memory and starts it. The **loader** follows these steps in UNIX systems:

1. Reads the executable file header to determine size of the text and data segments.
2. Creates an address space large enough for the text and data.
3. Copies the instructions and data from the executable file into memory.
4. Copies the parameters (if any) to the main program onto the stack.
5. Initializes the processor registers and sets the stack pointer to the first free location.
6. Branches to a start-up routine that copies the parameters into the argument registers and calls the main routine of the program. When the main routine returns, the start-up routine terminates the program with an `exit` system call.

Dynamically Linked Libraries

Virtually every problem in computer science can be solved by another level of indirection.

David Wheeler

The first part of this section describes the traditional approach to linking libraries before the program is run. Although this static approach is the fastest way to call library routines, it has a few disadvantages:

- The library routines become part of the executable code. If a new version of the library is released that fixes bugs or supports new hardware devices, the statically linked program keeps using the old version.
- It loads all routines in the library that are called anywhere in the executable, even if those calls are not executed. The library can be large relative to the program; for example, the standard C library is 2.5 MB.

dynamically linked libraries (DLLs) Library routines that are linked to a program during execution.

These disadvantages lead to **dynamically linked libraries (DLLs)**, where the library routines are not linked and loaded until the program is run. Both the program and library routines keep extra information on the location of nonlocal procedures and their names. In the original version of DLLs, the loader ran a dynamic linker, using the extra information in the file to find the appropriate libraries and to update all external references.

The downside of the initial version of DLLs was that it still linked all routines of the library that might be called, versus just those that are called during the running of the program. This observation led to the lazy procedure linkage version of DLLs, where each routine is linked only *after* it is called.

Like many innovations in our field, this trick relies on a level of indirection. Figure 2.23 shows the technique. It starts with the nonlocal routines calling a set of dummy routines at the end of the program, with one entry per nonlocal routine. These dummy entries each contain an indirect branch.

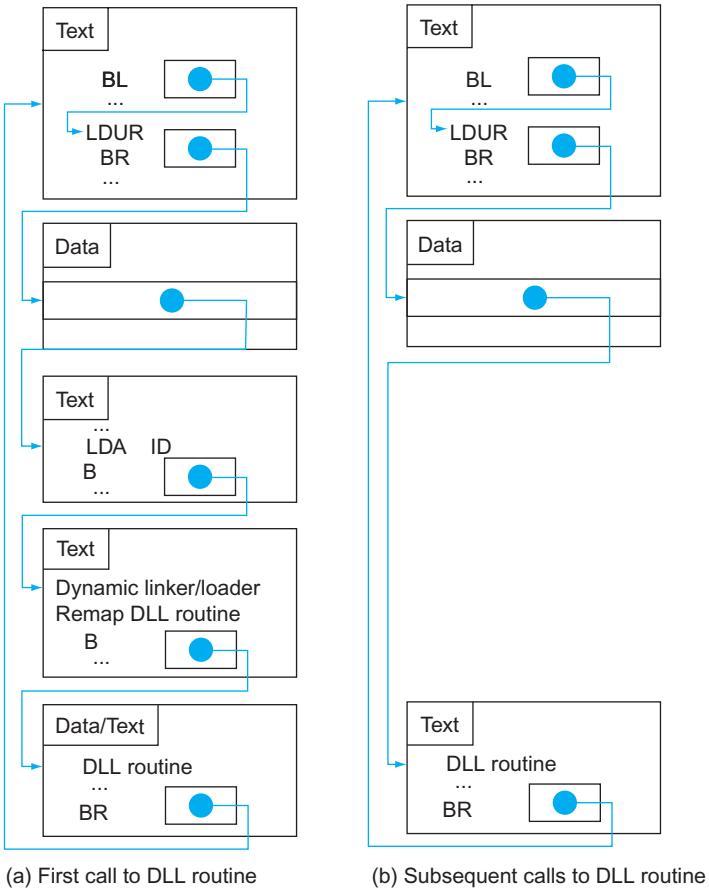


FIGURE 2.23 Dynamically linked library via lazy procedure linkage. (a) Steps for the first time a call is made to the DLL routine. (b) The steps to find the routine, remap it, and link it are skipped on subsequent calls. As we will see in Chapter 5, the operating system may avoid copying the desired routine by remapping it using virtual memory management.

The first time the library routine is called, the program calls the dummy entry and follows the indirect branch. It points to code that puts a number in a register to identify the desired library routine and then branches to the dynamic linker/loader. The linker/loader finds the wanted routine, remaps it, and changes the address in the indirect branch location to point to that routine. It then branches to it. When the routine completes, it returns to the original calling site. Thereafter, the call to the library routine branches indirectly to the routine without the extra hops.

In summary, DLLs require additional space for the information needed for dynamic linking, but do not require that whole libraries be copied or linked. They pay a good deal of overhead the first time a routine is called, but only a single indirect branch thereafter. Note that the return from the library pays no extra overhead. Microsoft's Windows relies extensively on dynamically linked libraries, and it is also the default when executing programs on UNIX systems today.

Starting a Java Program

The discussion above captures the traditional model of executing a program, where the emphasis is on fast execution time for a program targeted to a specific instruction set architecture, or even a particular implementation of that architecture. Indeed, it is possible to execute Java programs just like C. Java was invented with a different set of goals, however. One was to run safely on any computer, even if it might slow execution time.

Java bytecode
Instruction from an instruction set designed to interpret Java programs.

Figure 2.24 shows the typical translation and execution steps for Java. Rather than compile to the assembly language of a target computer, Java is compiled first to instructions that are easy to interpret: the **Java bytecode** instruction set (see **Section 2.15**). This instruction set is designed to be close to the Java language so that this compilation step is trivial. Virtually no optimizations are performed. Like the C compiler, the Java compiler checks the types of data and produces the proper operation for each type. Java programs are distributed in the binary version of these bytecodes.

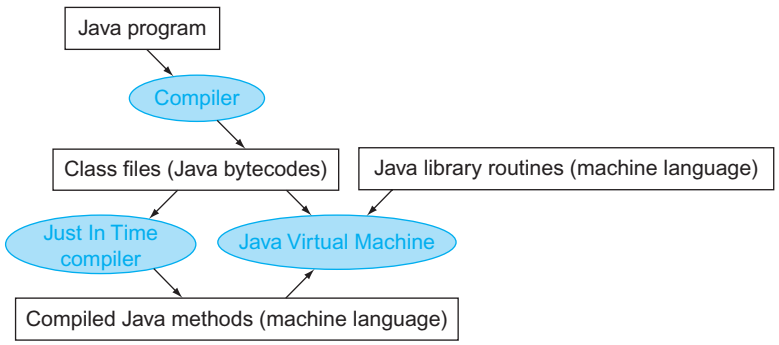


FIGURE 2.24 A translation hierarchy for Java. A Java program is first compiled into a binary version of Java bytecodes, with all addresses defined by the compiler. The Java program is now ready to run on the interpreter, called the *Java Virtual Machine* (JVM). The JVM links to desired methods in the Java library while the program is running. To achieve greater performance, the JVM can invoke the JIT compiler, which selectively compiles methods into the native machine language of the machine on which it is running.

A software interpreter, called a **Java Virtual Machine (JVM)**, can execute Java bytecodes. An interpreter is a program that simulates an instruction set architecture. For example, the ARMv8 simulator used with this book is an interpreter. There is no need for a separate assembly step since either the translation is so simple that the compiler fills in the addresses or JVM finds them at runtime.

Java Virtual Machine (JVM) The program that interprets Java bytecodes.

The upside of interpretation is portability. The availability of software Java virtual machines meant that most people could write and run Java programs shortly after Java was announced. Today, Java virtual machines are found in billions of devices, in everything from cell phones to Internet browsers.

The downside of interpretation is lower performance. The incredible advances in performance of the 1980s and 1990s made interpretation viable for many important applications, but the factor of 10 slowdown when compared to traditionally compiled C programs made Java unattractive for some applications.

To preserve portability and improve execution speed, the next phase of Java's development was compilers that translated *while* the program was running. Such **Just In Time compilers (JIT)** typically profile the running program to find where the "hot" methods are and then compile them into the native instruction set on which the virtual machine is running. The compiled portion is saved for the next time the program is run, so that it can run faster each time it is run. This balance of interpretation and compilation evolves over time, so that frequently run Java programs suffer little of the overhead of interpretation.

Just In Time compiler (JIT) The name commonly given to a compiler that operates at runtime, translating the interpreted code segments into the native code of the computer.

As computers get faster so that compilers can do more, and as researchers invent betters ways to compile Java on the fly, the performance gap between Java and C or C++ is closing.  **Section 2.15** goes into much greater depth on the implementation of Java, Java bytecodes, JVM, and JIT compilers.

Check Yourself

Which of the advantages of an interpreter over a translator was the most important for the designers of Java?

1. Ease of writing an interpreter
2. Better error messages
3. Smaller object code
4. Machine independence

2.13 A C Sort Example to Put it All Together

One danger of showing assembly language code in snippets is that you will have no idea what a full assembly language program looks like. In this section, we derive the LEGv8 code from two procedures written in C: one to swap array elements and one to sort them.

```
void swap(long long int v[], size_t k)
 {
   long long int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
 }
```

FIGURE 2.25 A C procedure that swaps two locations in memory. This subsection uses this procedure in a sorting example.

The Procedure `swap`

Let's start with the code for the procedure `swap` in Figure 2.25. This procedure simply swaps two locations in memory. When translating from C to assembly language by hand, we follow these general steps:

1. Allocate registers to program variables.
2. Produce code for the body of the procedure.
3. Preserve registers across the procedure invocation.

This section describes the `swap` procedure in these three pieces, concluding by putting all the pieces together.

Register Allocation for `swap`

As mentioned on page 100, the LEGv8 convention on parameter passing is to use registers `X0` to `X7`. Since swap has just two parameters, `v` and `k`, they will be found in registers `X0` and `X1`. The only other variable is `temp`, which we associate with register `X9` since swap is a leaf procedure (see page 113). This register allocation corresponds to the variable declarations in the first part of the swap procedure in Figure 2.25.

Code for the Body of the Procedure `swap`

The remaining lines of C code in swap are

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

Recall that the memory address for LEGv8 refers to the *byte* address, and so doublewords are really 8 bytes apart. Hence, we need to multiply the index `k` by 8 before adding it to the address. *Forgetting that sequential doubleword addresses differ by 8 instead of by 1 is a common mistake in assembly language programming.*

Hence, the first step is to get the address of `v[k]` by multiplying `k` by 8 via a shift left by 3:

```
LSL     X10, X1,#3        // reg X10 = k * 8
ADD     X10, X0,X10       // reg X10 = v + (k * 8)
                          // reg X10 has the address of v[k]
```

Now we load `v[k]` using `X10`, and then `v[k+1]` by adding 8 to `X10`:

```
LDUR    X9,  [X10,#0]     // reg X9 (temp) = v[k]
LDUR    X11, [X10,#8]     // reg X11 = v[k + 1]
                          // refers to next element of v
```

Next we store `X9` and `X11` to the swapped addresses:

```
STUR    X11, [X10,#0]     // v[k] = reg X11
STUR    X9,  [X10,#8]     // v[k+1] = reg X9 (temp)
```

Now we have allocated registers and written the code to perform the operations of the procedure. What is missing is the code for preserving the saved registers used within `swap`. Since we are not using saved registers in this leaf procedure, there is nothing to preserve.

The Full `swap` Procedure

We are now ready for the whole routine, which includes the procedure label and the return branch. To make it easier to follow, we identify in Figure 2.26 each block of code with its purpose in the procedure.

Procedure body

```
swap:  LSL    X10, X1,#3        # reg X10 = k * 8
       ADD    X10, X0,X10       # reg X10 = v + (k * 8)
                                # reg X10 has the address of v[k]
       LDUR   X9, [X10,#0]      # reg X9 (temp) = v[k]
       LDUR   X11,[X10,#8]      # reg X11 = v[k + 1]
                                # refers to next element of v
       STUR   X11,[X10,#0]      # v[k] = reg X11
       STUR   X9, [X10,#8]      # v[k+1] = reg X9 (temp)
```

Procedure return

```
       BR     LR                # return to calling routine
```

FIGURE 2.26 LEGv8 assembly code of the procedure `swap` in Figure 2.25.

```
void sort (long long int v[], size_t int n)
{
   size_t, j;
   for (i = 0; i < n; i += 1) {
        for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j +=1) {
            swap(v,j);
        }
   }
}
```

FIGURE 2.27 A C procedure that performs a sort on the array v.

The Procedure `sort`

To ensure that you appreciate the rigor of programming in assembly language, we'll try a second, longer example. In this case, we'll build a routine that calls the swap procedure. This program sorts an array of integers, using bubble or exchange sort, which is one of the simplest if not the fastest sorts. Figure 2.27 shows the C version of the program. Once again, we present this procedure in several steps, concluding with the full procedure.

Register Allocation for `sort`

The two parameters of the procedure `sort`, `v` and `n`, are in the parameter registers `X0` and `X1`, and we assign register `X19` to `i` and register `X20` to `j`.

Code for the Body of the Procedure `sort`

The procedure body consists of two nested *for* loops and a call to `swap` that includes parameters. Let's unwrap the code from the outside to the middle.

The first translation step is the first *for* loop:

```
for (i = 0; i <n; i += 1) {
```

Recall that the C *for* statement has three parts: initialization, loop test, and iteration increment. It takes just one instruction to initialize `i` to 0, the first part of the *for* statement:

```
MOV   X19, XZR  // i = 0
```

(Remember that `MOV` is a pseudoinstruction provided by the assembler for the convenience of the assembly language programmer; see page 129.) It also takes just one instruction to increment `i`, the last part of the *for* statement:

```
ADDI  X19, X19, #1  // i += 1
```

The loop should be exited if `i < n` is *not* true or, said another way, should be exited if `i ≥ n`. This test takes two instructions:

```
for1tst: CMP X19, X1    // compare X19 to X1(i to n)
         B.GE exit1     // go to exit1 if X19 ≥ X1 (i≥n)
```

The bottom of the loop just branches back to the loop test:

```
         B    for1tst   // branch to test of outer loop
exit1:
```

The skeleton code of the first *for* loop is then

```
        MOV X19, XZR       // i = 0
for1tst:CMP X19, X1        // compare X19 to X1 (i to n)
        B.GE exit1         // go to exit1 if X19 ≥ X1 (i≥n)
        ...
        (body of first for loop)
        ...
        ADDI X19, X19, #1  // i += 1
        B    for1tst       // branch to test of outer loop
exit1:
```

Voila! (The exercises explore writing faster code for similar loops.)

The second *for* loop looks like this in C:

```
for (j = i – 1; j >= 0 && v[j] > v[j + 1]; j – = 1) {
```

The initialization portion of this loop is again one instruction:

```
SUBI       X20, X19, #1 // j = i – 1
```

The decrement of `j` at the end of the loop is also one instruction:

```
SUBI       X20, X20, #1 // j – = 1
```

The loop test has two parts. We exit the loop if either condition fails, so the first test must exit the loop if it fails ($j < 0$):

```
for2tst: CMP X20,XZR    // compare X20 to 0 (j to 0)
         B.LT exit2     // go to exit2 if X20 < 0 (j < 0)
```

This branch will skip over the second condition test. If it doesn't skip, then `j ≥ 0`.